DevOps Shack

Cloud-Agnostic DevOps 50 Q&A

By DevOps Shack

Click here for DevSecOps & Cloud DevOps Course

DevOps Shack

Cloud-Agnostic DevOps: 50 Q&A

1) What is a load balancer and why do we need it in production?
Answer:
A load balancer is a network component that distributes incoming client traffic across multiple backend servers to ensure **high availability**, **fault tolerance**, **zero downtime**, and **efficient resource utilization**. Without a load balancer, a single backend handles all requests, which creates a **single point of failure**. If that server crashes, the application becomes unavailable. Load balancers also enable **horizontal scaling**, allowing new servers to be added or removed without affecting traffic. They improve performance through **optimized routing**, **health checks**, **connection reuse**, and sometimes **SSL/TLS termination**. In modern distributed systems, load balancers sit between clients and microservices and ensure traffic flows reliably even under failures.

2) Explain Layer-4 vs Layer-7 load balancing.

Answer:
Layer-4 load balancing happens at the **transport layer** (TCP/UDP). It routes packets based on IP address and port, without understanding HTTP, headers, cookies, or paths. It is extremely fast and suitable for low-latency and non-HTTP protocols.
Layer-7 load balancing happens at the **application layer**. It inspects the full HTTP request, including URL paths, headers, cookies, JWT tokens, or content type. It can do **path-based routing**, **host-based routing**, **A/B routing**, **blue-green**, and **canary deployments**. Because L7 LB understands the application, it can apply intelligent rules but has slightly more overhead compared to L4 LB.

3) What are health checks in load balancers and why are they essential?

Answer:
Health checks continuously monitor backend instances to verify they are functioning. A health

check can be a **TCP probe**, **HTTP GET**, **HTTPS check**, or **application-level custom probe**. If the LB sees continuous failures (e.g., HTTP 5xx), it marks the server **unhealthy** and automatically removes it from rotation. Once it recovers, the load balancer adds it back. Without health checks, a failed backend would continue receiving requests, leading to user errors and downtime.

4) What is SSL/TLS termination at a load balancer?

Answer:
TLS termination means the load balancer decrypts incoming HTTPS traffic and forwards it to the backend using either HTTP or HTTPS. This offloads heavy cryptographic operations from backend servers. The LB manages certificates, performs handshake, and re-encrypts if needed. This centralizes security and reduces certificate renewal complexity. Modern architectures, especially microservices, use L7 load balancers for TLS termination to reduce CPU load on app nodes.

5) What is SSL/TLS passthrough?

Answer:
TLS passthrough means the LB does **not decrypt** traffic. Instead, it forwards encrypted packets to the backend server, which performs the TLS handshake. This ensures **end-to-end encryption**, which is required in environments with strict compliance rules (PCI, HIPAA). However, the load balancer cannot apply L7 routing features (path routing, header inspection) because the data is encrypted.

6) Explain round-robin, weighted round-robin, and least-connections algorithms.

Answer:
Round-robin sends each request to the next server in sequence. It is simple and effective when servers have equal capacity.
Weighted round-robin gives higher-capacity servers more weight so they receive more traffic.
Least connections sends traffic to the server currently handling the fewest active connections. This is useful for long-running requests because a server may be overburdened even with equal capacity.

7) What is sticky session (session affinity) in load balancers?

Answer:
Sticky sessions ensure that a user's requests always go to the same backend server. This is useful for apps that store session data in server memory. However, it reduces high availability because if that server goes down, session data is lost. Modern best practice uses **distributed session stores** (Redis, Memcached) to avoid relying on sticky sessions.

8) What is connection draining (deregistration delay)?

Answer:
Connection draining allows a server to finish existing requests before being removed from rotation. Without it, active user sessions would break during deployments. Cloud load balancers use a configurable timeout (e.g., 300 seconds) to drain connections gracefully.

9) Why does Kubernetes use a load balancer even inside the cluster?

Answer:
Kubernetes services rely on **kube-proxy** to distribute traffic across Pods using round-robin or IPVS. For external traffic, Kubernetes uses **Ingress controllers** or **LoadBalancer services** to expose applications. This abstracts Pod churn and enables **auto-healing**, **rolling updates**, and **distributed routing** without requiring the client to keep track of pod IPs.

10) Explain global load balancers vs regional load balancers.

Answer:
Global load balancers distribute traffic across regions and use **Anycast IP**, DNS routing, latency-based routing, or geo-routing. They handle cross-continent failover.
Regional load balancers distribute traffic only within a specific datacenter or region. They do not offer global failover.

11) What is Anycast load balancing and how does it differ from DNS-based load balancing?

Answer:

Anycast assigns the same IP address to multiple servers worldwide. Routers automatically direct the client to the **nearest server** based on routing tables.
DNS-based load balancing uses DNS records to direct clients to different IPs but suffers from **DNS caching**, causing slow failover.

12) What is Layer-7 routing based on headers, cookies, and paths?

Answer:
L7 routing can inspect full HTTP requests. Example:
• `/api/v1/users` → user-service
• `/api/v1/orders` → order-service
• Header `X-Tenant-ID=AADI` → multi-tenant service
• Cookie value → A/B test groups
This enables microservices routing from a single domain.

13) What is blue-green deployment using load balancers?

Answer:
Two environments exist:
Blue = current production
Green = new version
Traffic switches instantly from Blue → Green using a load balancer. If issues appear, rollback is immediate by switching back.

14) What is canary deployment using load balancers?

Answer:
Canary deployment sends a small percentage of traffic (e.g., 5%) to new version servers. If metrics remain healthy, the LB gradually increases the percentage until full rollout. If issues occur, only a small portion of users are affected.

15) Explain rate limiting and throttling at load balancers.

Answer:
Rate limiting prevents abuse, DDoS, and excessive usage by limiting:
• requests per second
• connections per client
• bandwidth per client
Load balancers apply rate limiting globally or per API endpoint. Throttling protects backend servers from overload.

16) What is DNS and why is it critical?

Answer:
DNS resolves human-readable domain names into machine-readable IPs. Without DNS, users would need to remember IP addresses. It is a distributed hierarchical system involving root servers, TLDs, authoritative name servers, and resolvers. DNS underpins everything in modern systems—web browsing, APIs, email, microservices, and service discovery.

17) What is the difference between authoritative DNS and recursive DNS?

Answer:
Recursive DNS resolver queries multiple DNS servers on behalf of the client until it finds the final answer. ISPs, Google (8.8.8.8), and Cloudflare (1.1.1.1) run recursive resolvers.
Authoritative DNS stores the actual DNS records for your domain. It does not perform recursion; it only answers with the final record (A, CNAME, MX, TXT, etc.).

18) Explain DNS record types: A, AAAA, CNAME, TXT, MX, NS.

Answer:
A record: maps domain → IPv4
AAAA: maps domain → IPv6
CNAME: alias pointing to another hostname, not IP
TXT: arbitrary text, used for verification (SPF, DKIM, domain ownership)
MX: mail servers
NS: identifies authoritative DNS nameservers for domain

19) What is DNS TTL and why does it matter?

Answer:
TTL decides how long a DNS record stays cached in resolvers and browsers. High TTL reduces load on DNS servers and improves performance. Low TTL allows fast failover. For active migrations or blue-green deployments, TTL is often temporarily lowered (e.g., 60 seconds).

20) Explain DNS caching and how it affects failover.

Answer:
DNS caching happens at browsers, OS resolver cache, ISP resolver, and CDN. When a DNS record changes, many clients may still use cached values for minutes or hours. This slows down failover. TTL influences this behavior. Global load balancers overcome this issue using Anycast.

21) What is a CNAME vs A record?

Answer:
A record directly maps to an IP address.
A CNAME points to another hostname. CNAMEs allow DNS redirection without the need to update IPs everywhere. However, CNAME cannot be used at the root domain unless a DNS provider supports ALIAS or ANAME records.

22) What is split-horizon DNS?

Answer:
Split-horizon DNS provides different DNS responses depending on the source network.
Example:
• Internal users inside VPC resolve → private IP
• External users resolve → public IP
It is used for hybrid-cloud and VPN scenarios.

23) What is DNS propagation?

Answer:
DNS propagation is the time it takes for updated DNS records to replicate across global recursive resolvers and caches. It can take minutes to 48 hours depending on TTL.

24) What is DNS load balancing and how does it differ from L4/L7 load balancers?

Answer:
DNS load balancing returns multiple A records for the same domain. Traffic is distributed using client-side round-robin. But DNS does **not** perform health checks—clients may still reach dead servers until TTL expires. DNS load balancing is less intelligent than real load balancers.

25) What is an ALIAS/ANAME record? Why is it needed?

Answer:
Root domains cannot have CNAME records. ALIAS or ANAME records allow root domain mapping to another hostname, mainly used to point apex domain → cloud load balancer DNS name.

26) What is DNSSEC and why is it important?

Answer:
DNSSEC adds digital signatures to records to prevent DNS spoofing and cache poisoning. It ensures authenticity and integrity of DNS responses. Without DNSSEC, attackers can redirect traffic to fake servers.

27) What is reverse DNS?

Answer:
Reverse DNS resolves IP → domain, opposite of normal DNS. It is used in email servers, debugging, and credit card transaction verification.

28) What is wildcard DNS?

Answer:
Wildcard DNS resolves any undefined subdomain. Example:
`*.devopsshack.com` → resolves thousands of dynamic subdomains.
It's useful for multi-tenant SaaS.

29) How does DNS support multi-region redundancy?

Answer:
DNS can detect region health using:
• health checks (cloud DNS)
• geo-routing
• latency-based routing
If a region fails, DNS returns the backup region's IP.

30) Why is DNS not real-time for failover?

Answer:
Because DNS responses are cached, failover depends on TTL expiry. Large ISPs sometimes ignore low TTL values. Hence, DNS-only failover is not instantaneous.

31) What is TLS and why is it used?

Answer:
TLS (Transport Layer Security) encrypts communication between client and server. It ensures:
• confidentiality
• integrity
• authenticity
TLS prevents MITM, sniffing, replay attacks, DNS spoofing, and data theft. Modern internet uses TLS 1.3 due to improved security and faster handshake.

32) Explain the TLS handshake in complete detail.

Answer:

In TLS 1.3:

1. Client sends **ClientHello** with supported cipher suites and key share (ECDHE public key).
2. Server replies with **ServerHello**, its own key share, certificate, and certificate verify.
3. Both sides compute a shared secret using **ECDHE** (Diffie-Hellman).
4. HKDF expands the shared secret into symmetric session keys.
5. Client sends **Finished** encrypted with new key.
6. Server confirms and communication begins.
   TLS 1.3 reduces handshake to 1 round-trip and eliminates obsolete algorithms.

33) What is ECDHE and why is it widely used?

Answer:
ECDHE (Elliptic Curve Diffie-Hellman Ephemeral) enables secure key exchange over insecure networks. Both sides generate temporary keys, exchange public values, and derive the same session key without exposing private keys. Because keys are ephemeral, each connection gets a separate key — ensuring **Perfect Forward Secrecy**.

34) Explain Perfect Forward Secrecy in simple words.

Answer:
Even if an attacker steals the server's private key later, they **cannot** decrypt past traffic because each TLS session uses a unique ephemeral key that was never stored.

35) Difference between TLS 1.2 and TLS 1.3?

Answer:
TLS 1.3 removes insecure algorithms (RSA key exchange, CBC, SHA-1), reduces handshake complexity, uses only PFS-enabled cipher suites, and speeds up connection setup. TLS 1.2 still relies on multiple legacy algorithms, making it slower and less secure.

36) What is the importance of CA, Intermediate CA, and Root CA?

Answer:
Root CAs are trusted by browsers and OS. They issue intermediate CAs, which issue server certificates. This chain ensures trust and revocation control. Servers send the full certificate chain to clients so clients can validate authenticity.

37) What are SAN certificates and why are they common now?

Answer:
Subject Alternative Name (SAN) certificates allow multiple domains and subdomains in a single certificate. Browsers now require SAN instead of Common Name. SaaS platforms heavily use SAN for multi-domain support.

38) What is OCSP stapling?

Answer:
OCSP stapling allows the server to send a signed revocation status during handshake, avoiding slow lookups to CA servers. This improves performance and eliminates privacy leakage.

39) What is mutual TLS (mTLS) and when is it used?

Answer:
In mTLS, both client and server present certificates. It is used in service-to-service authentication, zero-trust networks, service mesh (Istio, Linkerd), and internal APIs. It prevents unauthorized microservices from calling sensitive internal APIs.

40) What is certificate pinning?

Answer:
Certificate pinning means hardcoding the server's public key or certificate hash in the client. Even if DNS is spoofed or CA is compromised, the client rejects impersonated servers.

41) What is autoscaling and why is it essential?

Answer:
Autoscaling dynamically adds or removes compute capacity based on load. It ensures applications run efficiently during peak load without manual intervention. Autoscaling reduces cost, improves resilience, and supports unpredictable traffic patterns.

42) Difference between vertical scaling and horizontal scaling.

Answer:
Vertical scaling increases resources (CPU/RAM) of a single instance — simple but has limits and downtime.
Horizontal scaling adds more instances — supports massive scale, fault tolerance, and zero downtime but requires load balancers.

43) Explain predictive vs reactive autoscaling.

Answer:
Reactive scaling responds to metrics (CPU > 70%).
Predictive scaling uses machine learning to forecast future load (e.g., weekly traffic patterns).
Predictive is used in e-commerce or gaming apps.

44) What triggers autoscaling?

Answer:
Autoscaling is triggered by metrics such as:
• CPU utilization
• memory usage
• request-per-second
• queue length
• latency
• custom business metrics
Autoscaling policies must avoid too frequent scaling to avoid oscillations.

45) What is cooldown period in autoscaling?

Answer:
Cooldown prevents autoscaler from adding/removing capacity too quickly. It stabilizes the system by giving enough time for changes to take effect. Without cooldown, autoscaling could thrash and cause instability.

46) How does autoscaling work in Kubernetes?

Answer:
Kubernetes supports:
HPA (Horizontal Pod Autoscaler): scales Pods based on CPU/memory/custom metrics.
VPA (Vertical Pod Autoscaler): adjusts Pod CPU/memory requests.
Cluster Autoscaler: scales the number of worker nodes.
Together, they allow workload and infrastructure autoscaling.

47) What are common autoscaling pitfalls?

Answer:
• Too slow scaling because metrics aren't set properly
• Scaling on CPU only when requests-per-second is more accurate
• Long startup times for backend apps
• Thrashing due to aggressive policies
• Wrong cooldown values
• Lack of warm instances
Autoscaling requires deep understanding of application behavior.

48) What is buffer capacity in autoscaling?

Answer:
Buffer capacity keeps a small number of idle servers/pods always running. This prevents sudden traffic spikes from overwhelming the system before new instances launch.

49) What is warm pool / pre-warmed capacity?

Answer:
Warm pool is a set of pre-initialized instances kept ready to join immediately. It avoids delays caused by instance boot time or app cold starts. Used in e-commerce flash sales and gaming events.

50) Explain autoscaling in microservices architectures.

Answer:
Each microservice scales independently based on its metrics. For example:
• user-service scales using requests per second
• order-service scales using queue depth
• payment-service scales using latency
Autoscaling ensures independent elasticity while minimizing cost. It also allows failure isolation so a spike in one service does not overload others.